ГУАП

КАФЕДРА № 43

ОТЧЕТ
ЗАЩИЩЕН С ОЦЕНКОЙ

ПРЕПОДАВАТЕЛЬ

| ассистент | | А. Э. Зянчурин |
|---|---|---|
| должность, уч. степень, звание | подпись, дата | инициалы, фамилия |

# ОТЧЕТ О ЛАБОРАТОРНОЙ РАБОТЕ №1

## Автоматизация технологического процесса промышленного производства программного обеспечения.

по курсу: Проектирование баз данных

РАБОТУ ВЫПОЛНИЛ

СТУДЕНТ

| гр. № 4434 | | |
|---|---|---|
| | подпись, дата | инициалы, фамилия |

Санкт-Петербург 2026

**СОДЕРЖАНИЕ**

ЗАДАНИЕ ..... 5

ВВЕДЕНИЕ ..... 6

1 Концепция и границы информационной системы супермаркета ..... 7

1.1 Описание предметной области ..... 7

1.2 Цель и постановка задачи ..... 7

1.2.1 Границы первой версии ..... 8

1.3 Словарь данных предметной области ..... 8

1.4 Нежелательные эффекты и конфликты ..... 11

1.5 Варианты использования ..... 11

1.6 Дерево функций ..... 15

1.7 Заключение и выводы ..... 16

2 Спецификация требований к информационной системе супермаркета ..... 17

2.1 Бизнес-требования ..... 17

2.2 Функциональные требования ..... 18

2.2.1 Ограничения и бизнес-правила ..... 19

2.2.2 Внешние интерфейсы ..... 20

2.3 Нефункциональные требования ..... 21

2.3.1 Информационные требования к UI ..... 22

2.3.2 Функциональные требования к UI ..... 22

2.4 Заключение и выводы ..... 22

3 Модель предметной области ..... 24

3.1 Диаграмма вариантов использования ..... 24

3.2 Диаграмма объектов ..... 26

3.3 Бизнес-процесс продажи на кассе ..... 27
3.4 Диаграмма состояний ..... 28
3.5 ER-модель ..... 29
3.6 Заключение и выводы ..... 29
3.7 Проверка машинной модели ..... 30
4 Архитектура программного обеспечения ..... 31
4.1 Вариант архитектуры и обоснование ..... 31
4.1.1 Модульная структура ..... 31
4.1.2 Компоненты и соединители ..... 32
4.2 Структуры распределения ..... 33
4.2.1 Размещение ..... 33
4.2.2 Реализация ..... 34
4.2.3 Распределение функций между участниками ..... 35
4.3 Надёжность и эксплуатационные границы ..... 35
4.4 Заключение и выводы ..... 36
5 Роли и защита ЛР №1 ..... 37
ЗАКЛЮЧЕНИЕ ..... 38
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ ..... 39

**ЗАДАНИЕ**

Разработка спецификации требований в соответствии с вариантом задания. Формализация ограничений и бизнес-правил предметной области.

Моделирование предметной области и архитектуры программного обеспечения с использованием нотаций UML, BPMN, ER-диаграмм и других языков моделирования по необходимости и согласованию с преподавателем.

Выполнение лабораторной работы предполагает коллективное взаимодействие в группе из трех человек в ролях: «Специалист по научноисследовательским и опытно-конструкторским разработкам», «Системный аналитик», «Архитектор программного обеспечения».

# ВВЕДЕНИЕ

Работа посвящена требованиям и проектированию информационной системы супермаркета по варианту 36. Концепция, ограничения, модели и архитектура задают проверяемый контракт будущей реализации. Исполняемый прикладной код на этом этапе не разрабатывается.

Основание выполнения — соответствующая методичка [1]. Порядок оформления сверяется с нормативной документацией ГУАП [2].

# 1 Концепция и границы информационной системы супермаркета

## 1.1 Описание предметной области

Рассматривается один супермаркет с общим каталогом и остатком товаров, несколькими рабочими местами кассиров и рабочим местом менеджера. Менеджер ведёт карточки товаров, задаёт розничные цены, регистрирует поступления и корректировки остатка. Кассир находит товар по названию или полному штрихкоду, формирует чек и подтверждает получение наличных. Аудитор просматривает каталог и историю продаж.

Объект автоматизации — согласованная обработка товарных данных и кассовых продаж. В исходном организационном сценарии каталог и складской учёт ведутся раздельно. Кассир может видеть устаревшую цену, а две кассы — одновременно считать доступным последний экземпляр товара. Это проектная постановка проблемы, а не результат проведённого обследования реального магазина.

Внутренние участники: менеджер, кассир, аудитор, администратор БД. Покупатель участвует в бизнес-процессе, но не входит в интерфейс сотрудников. Банк и фискальный оператор находятся за границами учебного продукта. Учебный чек фиксирует внутреннюю продажу и не является интеграцией с кассовым фискальным устройством.

## 1.2 Цель и постановка задачи

Цель — создать единый источник товарных данных и обеспечить согласованность чека с движением остатка при параллельной работе кассиров.

Задачи проектирования:

1. Зафиксировать словарь сущностей, диапазоны значений и правила изменения данных.
2. Определить пользовательские сценарии и доступ к ним по ролям.
3. Исключить отрицательные остатки, повторное списание и изменение завершённых чеков.

4. Спроектировать поиск и ограниченную постраничную выдачу для каталога около 100 тысяч объектов.

5. Разделить интерфейс, бизнес-логику и хранение данных в клиент-серверной архитектуре.

Ожидаемый эффект: кассир использует актуальный единый каталог; цена и наименование сохраняются в строке чека; продажа либо целиком фиксируется вместе со списанием, либо не изменяет состояние БД. Измеримые критерии приёмки содержатся в спецификации.

### 1.2.1 Границы первой версии

Включаются: вход сотрудников, каталог, поиск, управление карточками, архивирование, корректировка остатков с основанием, черновик чека, оплата наличными, отмена черновика, просмотр истории и настройка инфраструктуры. Рабочая единица количества — штука; фасованные продукты продаются упаковками.

За границами версии: весовые товары с дробным количеством, возвраты оплаченных чеков, скидочные программы, эквайринг, фискализация, поставщики и закупочные заказы, учёт партий и сроков годности, несколько магазинов, автономная касса при потере сети. Такие расширения требуют изменения требований и схемы данных.

## 1.3 Словарь данных предметной области

Запись составляется знаком +; необязательное поле заключено в круглые скобки; 1:100{Строка чека} означает от одной до ста строк. Примитивы таблицы 1 определяют диапазоны структур таблицы 2.

Таблица 1 — Элементы данных

| Элемент данных | Описание | Структура или тип данных | Количество | Значение |
|---|---|---|---|---|
| Чек | Документ продажи товаров покупателю на кассе супермаркета | Идентификатор чека + Ключ запроса + Хеш корзины + Идентификатор кассира + Статус чека + Время создания + (Время оплаты) + 1:100{Строка чека} | — | Итоговая сумма вычисляется по строкам чека |
| Строка чека | Запись о конкретной товарной позиции, входящей в состав чека | Идентификатор чека + Идентификатор товара + Название товара + Штрихкод товара + Количество в строке + Цена товара + Сумма строки | — | Фиксированный снимок цен на момент оформления продажи |
| Товар | Карточка товарной номенклатуры в каталоге и складском учёте | Идентификатор товара + Штрихкод товара + Название товара + Категория товара + Цена товара + Остаток товара + Признак активности + Версия карточки + Время изменения | — | Единица торгового и складского учёта |
| Движение остатка | Документ аудита изменения складского остатка товара | Идентификатор движения + Идентификатор товара + Идентификатор сотрудника + (Идентификатор чека) + Изменение остатка + Основание изменения + Время операции | — | Фиксирует списание, продажу или оприходование |
| Сотрудник | Учётная запись пользователя информационной системы супермаркета | Идентификатор сотрудника + Логин + Имя сотрудника + Хеш пароля + Роль сотрудника + Признак активности | — | Идентификация и разграничение прав доступа |
| Сеанс работы | Активная сессия авторизованного сотрудника в системе | Хеш токена сессии + Идентификатор сотрудника + Время истечения | — | Срок жизни токена — 8 часов |
| Идентификатор | Уникальный системный номер сущности (чека, товара, сотрудника, проводки) | Положительное целое (BIGINT) | 19 | Генерируемый СУБД порядковый номер, начиная с 1 |
| Штрихкод товара | Уникальный штриховой идентификатор потребительской упаковки | Цифровые символы | 14 | 8–14 ASCII-цифр стандартов EAN-8, EAN-13, ITF-14; уникален |
| Название товара | Торговое наименование товара для чека, ценника и каталога | Текстовые символы Unicode | 160 | 2–160 символов после удаления пробелов по краям |
| Категория товара | Товарная группа для классификации в каталоге | Текстовые символы Unicode | 60 | 2–60 символов справочника категорий |
| Цена товара | Розничная цена за одну единицу товара | Десятичное число | 10 | 0,01–999 999,99 руб.; точность 2 знака после запятой |

| Элемент данных | Описание | Структура или тип данных | Количество | Значение |
|---|---|---|---|---|
| Остаток товара | Фактическое количество упаковок товара на складе и в зале | Целое неотрицательное | 10 | 0–1 000 000 000 шт. |
| Количество в строке | Число единиц товара, приобретаемое по чеку | Целое положительное | 5 | 1–10 000 шт.; товар включается в чек не более одного раза |
| Сумма строки | Стоимость товарной позиции с учётом цены и количества | Десятичное число | 12 | Произведение: Цена товара × Количество в строке |
| Изменение остатка | Величина корректировки, списания или поступления товара | Целое со знаком | 10 | Для ручной операции от −1 000 000 до +1 000 000, кроме 0 |
| Основание изменения | Причина ручной корректировки остатка менеджером | Текстовые символы Unicode | 200 | 5–200 символов (например, «Акт списания боя № 14») |
| Статус чека | Текущее состояние документа продажи в жизненном цикле | Символьный код (перечисление) | 10 | draft (черновик), paid (оплачен), cancelled (отменён) |
| Ключ запроса | Клиентский идентификатор сетевой попытки создания чека | Символы UUID | 36 | Стандартный UUID v4 для защиты от повторных списаний |
| Хеш корзины | Контрольная сумма состава товаров для выявления подмены | Шестнадцатеричные символы | 64 | Хеш-код SHA-256 от отсортированных позиций корзины |
| Хеш пароля | Криптографический хеш пароля с индивидуальной солью | Текстовые символы | 128 | Алгоритм scrypt; открытый пароль в БД не хранится |
| Хеш токена сессии | Хеш случайного токена авторизации для сеанса работы | Шестнадцатеричные символы | 64 | Хеш SHA-256; оригинальный токен хранится у клиента |
| Логин | Уникальное имя учётной записи для аутентификации | Буквенно-цифровые символы | 50 | 3–50 латинских символов, цифры, знак _ |
| Имя сотрудника | Отображаемое в интерфейсе и документах имя работника | Текстовые символы Unicode | 100 | Фамилия и имя сотрудника от 2 до 100 символов |
| Роль сотрудника | Уровень служебных полномочий в системе | Символьный код (перечисление) | 10 | manager (менеджер), cashier (кассир), auditor (аудитор) |
| Признак активности | Флаг доступности записи в операционных процессах | Логический (BOOLEAN) | 1 | true (активен), false (в архиве / заблокирован) |
| Версия карточки | Номер ревизии для оптимистической блокировки карточки | Целое положительное | 10 | Инкрементируется СУБД при каждом изменении карточки |

| Элемент данных | Описание | Структура или тип данных | Количество | Значение |
|---|---|---|---|---|
| Время | Метка времени фиксации операции или изменения документа | Дата и время с часовым поясом | 35 | Формат ISO 8601 (хранение в типе PostgreSQL timestamptz) |

### 1.4 Нежелательные эффекты и конфликты

Обозначения из методички: + — конфликт, − — отсутствие конфликта, · — связь не рассматривается. Сравниваются подсистемы исходного процесса.

Таблица 3 — Матрица взаимодействий

| Элемент | Каталог цен | Касса А | Касса Б | Остаток | История |
|---|---|---|---|---|---|
| Каталог цен | · | + | + | – | + |
| Касса А | · | · | + | + | + |
| Касса Б | · | · | · | + | + |
| Остаток | · | · | · | · | + |
| История | · | · | · | · | · |

### 1.5 Варианты использования

При выполнении анализа информации о предметной области необходимо сопоставлять различные её представления на предмет обнаружения пробелов, ошибок и противоречий. Одним из наиболее строгих и точных способов поиска недостающих элементов системы является построение матрицы CRUD (*Create* — создание, *Read* — чтение, *Update* — обновление, *Delete* — удаление).

Матрица CRUD позволяет соотнести действия предполагаемой информационной системы с элементами и структурами данных, что даёт конкретное представление о том, где и как каждый элемент создаётся, считывается, модифицируется или удаляется.

**Состав вариантов использования системы супермаркета:**

UC01 «Аутентификация и управление сеансом» (Актор: *Сотрудник*) — проверка логина и пароля, выдача токена доступа, контроль активности учетной записи и отзыв сеанса при выходе.

UC02 «Поиск и просмотр каталога товаров» (Актор: *Сотрудник*) — поиск по штрихкоду, подстроке наименования, фильтрация по категориям и постраничный просмотр карточек.

UC03 «Ведение карточек товаров и архива» (Актор: *Менеджер*) — заведение новой номенклатуры, редактирование реквизитов (название, категория, розничная цена) и перевод неактуальных товаров в архив.

UC04 «Корректировка складского остатка» (Актор: *Менеджер*) — фиксация ручных изменений остатка (оприходование, инвентаризация, списание брака) с обязательным указанием основания.

UC05 «Формирование черновика чека» (Актор: *Кассир, Менеджер*) — добавление отсканированных товаров в корзину с фиксацией мгновенного снимка розничных цен.

UC06 «Подтверждение оплаты и закрытие чека» (Актор: *Кассир*, *Менеджер*) — приём наличных денежных средств, повторная проверка доступности остатков, атомарное списание товаров и перевод чека в статус оплаченного.

UC07 «Отмена черновика чека» (Актор: *Кассир*, *Менеджер*) — аннулирование неоплаченного документа продажи покупателем без изменения товарных остатков.

UC08 «Просмотр журнала продаж и деталей чеков» (Актор: *Сотрудник*) — аудит совершённых операций (кассир имеет доступ только к собственным чекам текущей смены; менеджеру и аудитору доступен полный журнал магазина).

UC09 «Администрирование инфраструктуры БД» (Актор: *Администратор БД*) — создание резервных копий, контроль целостности и управление учётными записями СУБД вне пользовательского веб-интерфейса.

Сопоставление вариантов использования с ключевыми информационными сущностями представлено в таблице 4.

Таблица 4 — Матрица CRUD

| Вариант использования | Чек | Строка чека | Товар | Движение остатка | Сотрудник | Сеанс работы |
|---|---|---|---|---|---|---|
| UC01 Вход / выход из системы | — | — | — | — | R | C, U, D |
| UC02 Поиск и просмотр товаров | — | — | R | — | — | R |
| UC03 Ведение карточек товаров | — | — | C, U | — | — | R |
| UC04 Корректировка остатка | — | — | U | C | — | R |
| UC05 Создание черновика чека | C | C | R | — | — | R |
| UC06 Оплата наличными и списание | U | — | U | C | — | R |
| UC07 Отмена черновика продажи | U | — | — | — | — | R |
| UC08 Просмотр истории чеков | R | R | R | R | R | R |
| UC09 Обслуживание базы данных | R | R | R | R | R | — |

**Анализ матрицы CRUD на предмет пропущенных требований:**

Анализ заполнения таблицы 4 в соответствии с рекомендациями методических указаний позволяет сделать следующие выводы:

Отсутствие операций удаления (D) в колонке «Товар»: Ни один вариант использования не выполняет физического удаления (Delete) карточки товара. Это является осознанным архитектурным решением: удаление товара из БД привело бы к нарушению целостности внешних ключей

(Foreign Keys) в исторических чеках прошлых периодов. Вместо удаления реализован перевод карточки в архив (операция Update флага is_active = false).

Отсутствие операций обновления (U) и удаления (D) в колонке «Строка чека»:

Строки чека создаются единожды (Create) при фиксации черновика. Возможность их последующей модификации через API отсутствует для предотвращения подмены цен и расхождений итоговых сумм. При ошибке кассира черновик целиком отменяется (UC07) и создаётся заново.

Отсутствие операций обновления (U) и удаления (D) в колонке «Движение остатка»:

Таблица движений остатка выполняет функцию строгого журнала аудита (Append-only log). Любое изменение остатка (как продажа, так и инвентаризация) порождает новую проводку (Create), а ранее созданные записи неизменяемы.

Контроль создания данных:

Все сущности имеют парные операции создания (C) и чтения (R), что подтверждает отсутствие «висячих» данных, которые система читает, но не имеет механизмов их инициализации.

## 1.6 Дерево функций

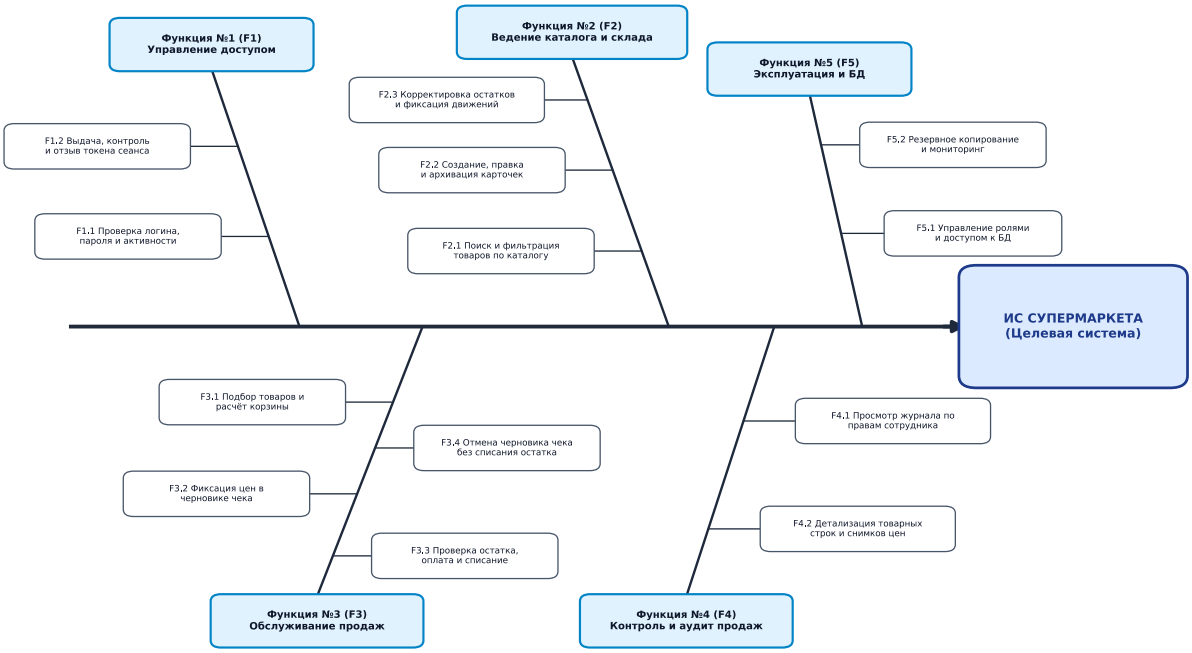



Рисунок 1 — Дерево функций ИС супермаркета

Каждая ветвь дерева функций непосредственно направлена на реализацию вариантов использования (UC) и нейтрализацию нежелательных эффектов (конфликтов C1C1C1–C7C7C7):

- F1 реализует UC01 и устраняет конфликт C7C7C7 (несанкционированный доступ кассира к правке цен и остатков);
- F2 реализует UC02–UC04 и устраняет конфликты C1C1C1 (рассинхронизация цен при подборе) и C6C6C6 (перегрузка клиентского интерфейса при больших объемах номенклатуры);
- F3 реализует ключевые транзакционные сценарии UC05–UC07 и разрешает конфликты C2C2C2 (продажа последнего товара двумя кассами одновременно), C4C4C4 (потеря сетевого пакета при подтверждении оплаты) и C5C5C5 (рассинхронизация списания остатка и сохранения чека);
- F4 реализует сценарий аудита UC08 и устраняет конфликт C3C3C3 (изменение карточки товара задним числом искажает архивные чеки);

- F5 реализует UC09 и обеспечивает отказоустойчивость инфраструктуры хранения данных.

### 1.7 Заключение и выводы

1. Центральная сущность продажи — чек со строками, а не отдельная запись товара.
2. Историческую цену необходимо отделить от текущей цены каталога.
3. Управление остатком и фиксация оплаты должны иметь общую транзакционную границу.
4. Объём 100 тысяч объектов требует ограничения выдачи, но сам по себе не обосновывает распределённую БД.
5. Границы учебной версии позволяют проверить основные правила без внешних платёжных и фискальных интеграций.

## 2 Спецификация требований к информационной системе супермаркета

Версия требований 1.0. Предметная область — вариант 36. Статус документа на этапе ЛР №1: проектная спецификация. Приоритет Must обязателен для версии, Should — целевой критерий качества. Стабильность «высокая» относится к бизнес-правилам, «средняя» — к численным целям и интерфейсу.

Требования структурированы по заданию лабораторной работы и сформулированы как проверяемые результаты для стейкхолдеров и системы; дополнительный ориентир по качеству требований — ISO/IEC/IEEE 29148:2018 [3].

### 2.1 Бизнес-требования

Таблица 5 — Требования к системе в целом

| ID | Требование | Приоритет и проверка |
|---|---|---|
| BR01 | Общие каталог, остатки и история одного супермаркета; три уровня: Web-клиент, сервер приложений, БД | Must, высокая; архитектурная проверка |
| BR02 | Обычный режим — подключение к серверу; при отказе сети новые продажи не подтверждаются локально | Must, высокая; отключение сети при оплате |
| BR03 | Пользователи продукта: кассир, менеджер, аудитор; эксплуатация — администратор БД | Must, высокая; таблица прав и отрицательные проверки |
| BR04 | Персонал смены: до 10 кассиров (базовые навыки работы с ПК, инструктаж ≤1 ч) и менеджер зала; сменный режим до 12 ч; администрирование СУБД требует навыков PostgreSQL/Linux | Must, высокая; юзабилити-тест без спецобучения и регламент эксплуатации |
| BR05 | Подтверждённая продажа сохраняет исторические реквизиты и не списывает остаток повторно | Must, высокая; повтор и конкурентные запросы |
| BR06 | Работа на стационарной кассе через браузер; интерфейс также доступен с узкого экрана | Must, средняя; 390, 768 и 1440 px |

| ID | Требование | Приоритет и проверка |
|---|---|---|
| BR07 | Резервное копирование администратором и проверяемое восстановление в отдельную БД | Must, высокая; протокол восстановления |
| BR08 | Обслуживание и обновление по версиям; перед изменением схемы — резервная копия; хранение кода в Git | Must, высокая; регламент выпуска |
| BR09 | Секреты вне Git; браузер не получает учётные данные PostgreSQL | Must, высокая; анализ сборки и конфигурации |
| BR10 | Открытые форматы обмена JSON/HTTP, SQL и текстовые исходники моделей | Must, высокая; проверка интерфейсов |

Транспортабельность для подвижных АС не применяется: система стационарная. Специальная защита от физических воздействий обеспечивается условиями размещения оборудования, ИБП и копиями; аппаратная сертификация не входит в проект. Патентное исследование не проводилось; при распространении проверяются лицензии зависимостей. Учебная демонстрация не заявляет соответствие требованиям реальной фискализации.

## 2.2 Функциональные требования

Таблица 6 — Проверяемые функции

| ID / UC | Обязательное поведение | Критерий приёмки |
|---|---|---|
| FR01 / UC01 | Вход по паролю, проверка активности, срок сеанса 8 ч, выход с отзывом токена | Неверный пароль и истёкший токен дают 401; отозванный токен не работает |
| FR02 / UC02 | Поиск подстроки названия от 3 символов; полный штрихкод; точный фильтр категории; выдача по ID | limit 1–100; за пределом 422; одинаковый ID не повторяется при последовательном обходе стабильного набора |
| FR03 / UC03 | Создание и изменение карточки менеджером; архивирование вместо физического удаления | Дубликат штрихкода 409; кассир получает 403; устаревшая версия 409 |
| FR04 / UC04 | Корректировка остатка со знаком и обязательным основанием | При нехватке остатка 409; успешная операция создаёт движение и меняет остаток атомарно |

| ID / UC | Обязательное поведение | Критерий приёмки |
|---|---|---|
| FR05 / UC05 | Создание чека с 1–100 различными товарами и снимками цен | Пустой состав, повтор товара, дробное количество и неизвестные поля дают 422; отсутствующий товар 409 |
| FR06 / UC06 | Оплата наличными подтверждается сотрудником; цены берутся из строк чека; все остатки проверяются повторно | В конкурентной продаже последнего товара успешно завершается ровно одна оплата |
| FR07 / UC07 | Отмена только draft; повтор отмены cancelled возвращает тот же результат | Оплаченный чек отменить нельзя: 409; остаток при отмене не меняется |
| FR08 / UC08 | Список и детализация чеков с ограничением доступа | Кассир видит свои чеки; чужой ID возвращает 404; аудитор не может оплатить чек |
| FR09 / UC05–06 | Идемпотентность создания и оплаты | Повтор UUID с тем же составом возвращает тот же чек; другой состав/автор — 409; повтор pay не создаёт второе движение |
| FR10 / UC09 | Эксплуатационная проверка API и соединения с БД | Health возвращает ok только при доступной БД |

### 2.2.1 Ограничения и бизнес-правила

1. R01: price > 0; деньги вычисляются десятичной арифметикой. В JSON суммы передаются строками, чтобы не зависеть от точности IEEE-754.
2. R02: остаток и количество целые; stock ≥ 0; весовой товар предварительно фасуется и учитывается упаковками.
3. R03: один штрихкод соответствует одной карточке, включая архив; повторное использование кода другой карточкой запрещено.
4. R04: черновик не резервирует товар. Между созданием и оплатой наличие может измениться; оплата повторно проверяет остаток и активность.
5. R05: цена фиксируется при создании черновика и сохраняется до завершения; последующие изменения каталога не меняют этот документ.
6. R06: total = Σ(quantity × unit_price); клиент не передаёт итог, цену строки, статус, автора и время оплаты.
7. R07: оплата, движения и уменьшение остатков составляют одну транзакцию; при любой ошибке всё откатывается.

8. R08: допустимы только draft → paid и draft → cancelled; завершённые документы неизменны. Повтор запроса завершения читает существующий результат.

9. R09: строки чека не удаляются и не редактируются через публичный API; для исправления состава черновик отменяют и создают новый.

10. R10: карточки архивируются; ссылки из чеков и движений сохраняются. Физическое удаление исторических данных не является пользовательской функцией.

11. R11: ключ создания уникален; цена не входит в клиентский запрос, хеш определяется по отсортированным ID и количествам. Ключ повторяется при неизвестном результате сетевого запроса.

12. R12: кассир работает со своими чеками, менеджер — со всеми, аудитор выполняет только чтение. Администратор БД обслуживает пользователей вне API.

### 2.2.2 Внешние интерфейсы

Браузер обращается к серверу по HTTP/JSON под /api, в эксплуатации — через HTTPS на внешнем доверенном прокси. API использует Pydantic для схем, PostgreSQL — для транзакций и ограничений. Штрихкод вводится с клавиатуры; USB-сканер может работать как клавиатурное устройство. В первой версии автоматическое управление устройствами не требуется.

Ошибки: 401 — нет действующего сеанса; 403 — нет права; 404 — объект отсутствует или недоступен; 409 — конфликт данных/состояния; 422 — ошибка входных данных; 503 — временная недоступность. Сообщения не раскрывают SQL и пароли. Полный каталог не загружается для локальной фильтрации.

## 2.3 Нефункциональные требования

Таблица 7 — Качество и условия проверки

| ID | Цель | Измерение и границы |
|---|---|---|
| NFR01 | Каталог 100 000 товаров, не менее 100 000 чеков с несколькими строками | Синтетический набор в отдельной тестовой БД; реальные объёмы индексов фиксируются отдельно |
| NFR02 | p95 GET каталога ≤500 мс при 20 конкурентных пользователях после прогрева | Цель для VM 4 vCPU, 8 ГБ RAM, SSD; отдельное измерение полного API, а не только SQL EXPLAIN |
| NFR03 | p95 оплаты ≤1 с без конкуренции за один товар | Цель для того же стенда; конфликтующие продажи могут ожидать блокировку до 3 с и получить 503 |
| NFR04 | Атомарность продаж и отсутствие отрицательных остатков | Обязательный интеграционный тест на настоящем PostgreSQL |
| NFR05 | Ограниченное потребление ресурсов | Максимум 100 строк выдачи и 100 строк чека; пул до 10 соединений на процесс API; тело HTTP до 128 КБ |
| NFR06 | Разграничение доступа | Отрицательные проверки всех изменяющих маршрутов; роль API без DDL и без удаления товаров/чеков |
| NFR07 | Сохранность при аварии | Транзакции PostgreSQL/WAL; целевые RPO 24 ч, RTO 60 мин при ежедневной копии и проведённом восстановлении |
| NFR08 | Воспроизводимость | Зафиксированные зависимости Python/JS, контейнерные файлы и сценарии запуска; изменение версии проходит проверку |
| NFR09 | Доступность интерфейса | Клавиатура, подписанные поля, видимый фокус, состояния загрузки/ошибки/пустого результата, отсутствие выхода содержимого за экран |
| NFR10 | Наблюдаемость | HTTP-журналы и health; секреты и тело входа не пишутся в журналы; постоянные ошибки исследуются по логам контейнера |

NFR02–03 и RPO/RTO — цели приёмки, не гарантии и не результаты измерений на этапе проектирования. В учебной конфигурации один экземпляр БД и API; высокая доступность и автоматическое переключение не заявлены.

### 2.3.1 Информационные требования к UI

UI01: карточка показывает название, штрихкод, категорию, цену за штуку, остаток и доступность добавления. UI02: корзина показывает название, количество, цену и предварительный итог; после создания чека отображается серверная сумма. UI03: история показывает ID, время, статус; детализация — строки и итог. UI04: профиль показывает сотрудника и роль; пароль не отображается открытым текстом.

### 2.3.2 Функциональные требования к UI

UI05: поиск имеет задержку 300 мс и отменяет устаревший запрос; ответ старого запроса не заменяет новый. UI06: переход вперёд/назад не накапливает весь каталог. UI07: во время отправки кнопка блокируется; при неизвестном исходе создания корзина фиксируется до повтора с тем же ключом. UI08: подтверждение наличных явно отделено от формирования чека. UI09: диалог редактирования управляет фокусом и закрывается Escape; ошибки не очищают введённые поля. UI10: PropTypes проверяют контракты компонентов при разработке, а проверка ответа API действует также в production.

## 2.4 Заключение и выводы

1. Требования сформулированы через наблюдаемые результаты и коды ошибок.
2. Сохранение истории и безопасная повторная отправка обязательны даже в учебной версии.
3. Пользовательские роли и роли СУБД выполняют разные задачи и проверяются отдельно.

4. Производительность подлежит измерению на указанном объёме, а не объявляется доказанной наличием индекса.

# 3 Модель предметной области

## 3.1 Диаграмма вариантов использования

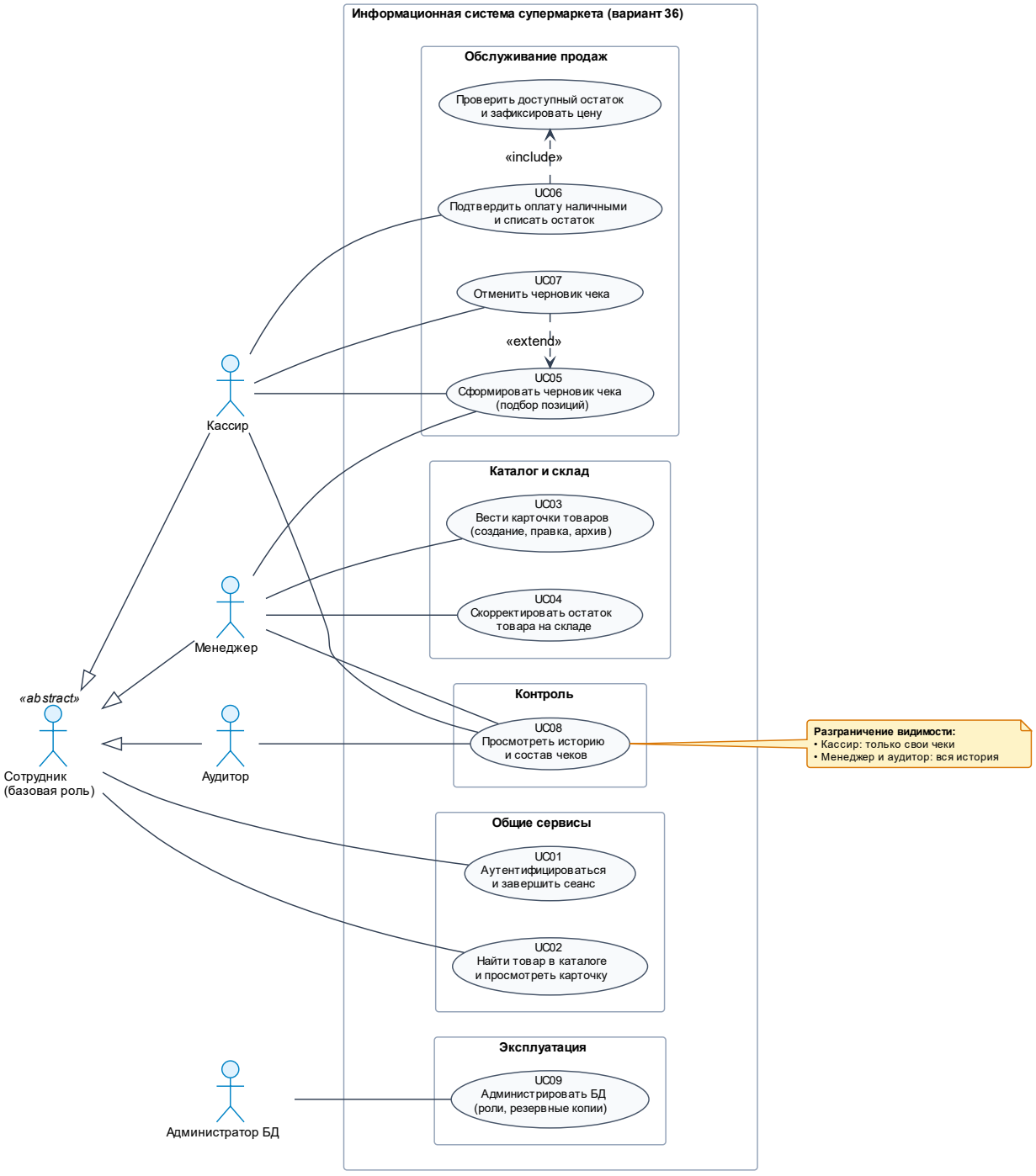



Рисунок 2 — диаграмма use case

Чтобы убрать повторяющиеся связи, актор «Сотрудник» объединяет три роли кассира, менеджера и аудитора; отдельный актор показан для администратора БД. Разрешённые роли указаны прямо в вариантах

использования: ведение товара и корректировка остатка доступны менеджеру, чеки ограничены ролью. Сценарии продажи декомпозированы в соответствии с их логикой: базовый вариант использования UC05 «Сформировать черновик чека» расширяется сценарием UC07 при отмене покупки, а подтверждение оплаты UC06 включает атомарную процедуру списания остатков. Детальные проверяемые критерии приёмки и бизнес-правила для каждого прецедента специфицированы в разделе 2 (таблицы 6 и 7). Аутентификация в системе является обязательным предусловием для выполнения защищённых действий. Нотация и отношения вариантов использования сверены с UML 2.5.1 [4].

Основной сценарий UC05–06: кассир входит, находит товар, добавляет количество, отправляет корзину, проверяет серверный итог, получает наличные, подтверждает оплату, видит завершённый чек. Предусловия: активная учётная запись, допустимые количества, товары существуют. Постусловия: чек paid, остатки уменьшены, создано по движению на товар.

Альтернативы: товара недостаточно — черновик остаётся draft; кассир отменил покупку — cancelled; сеть потеряла ответ — повтор того же действия; товар архивирован до оплаты — отказ и последующая отмена черновика. Оплата cancelled и отмена paid запрещены.

### 3.2 Диаграмма объектов

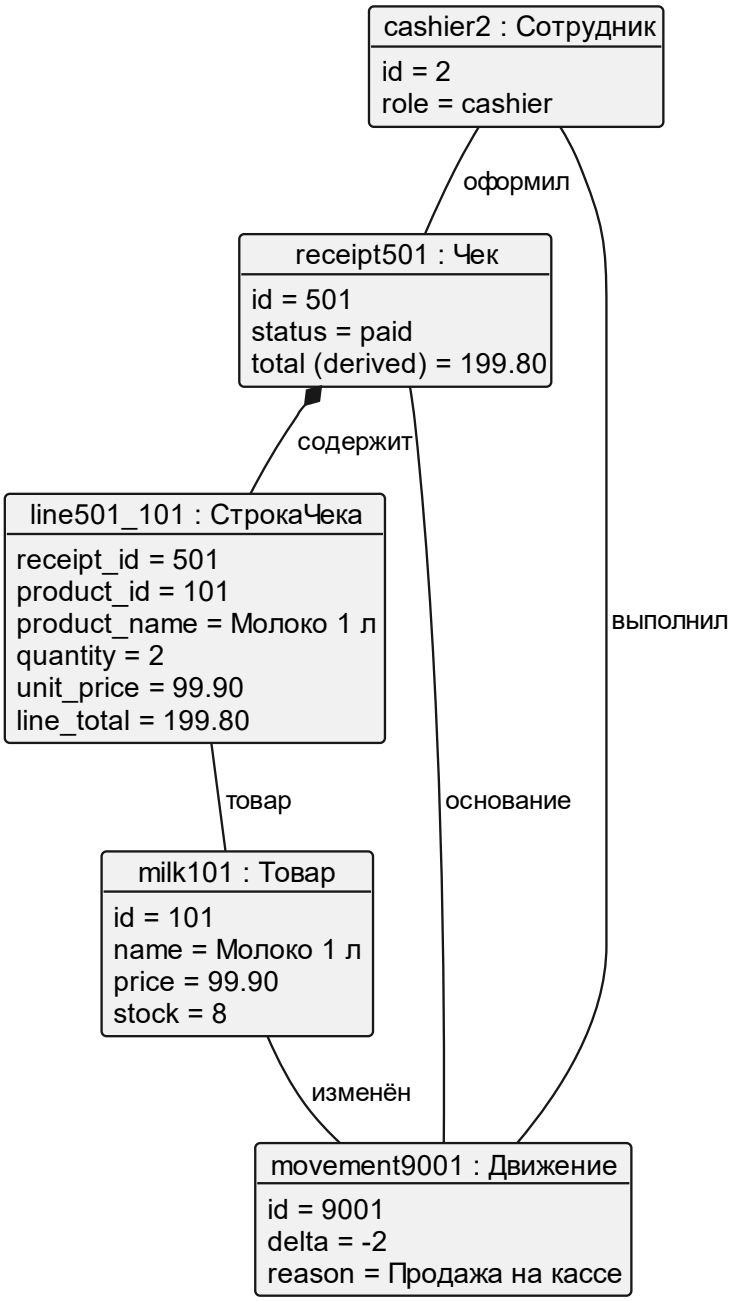



Рисунок 3 — диаграмма объектов

Изображены экземпляры после продажи двух упаковок молока: товар id=101, кассир id=2, чек id=501, строка (501,101) и движение id=9001. Цена 99,90, количество 2, итог 199,80. Остаток товара после списания — 8. Это снимок объектов, не диаграмма классов. Примерные ID не задают начальные данные реализации.

### 3.3 Бизнес-процесс продажи на кассе

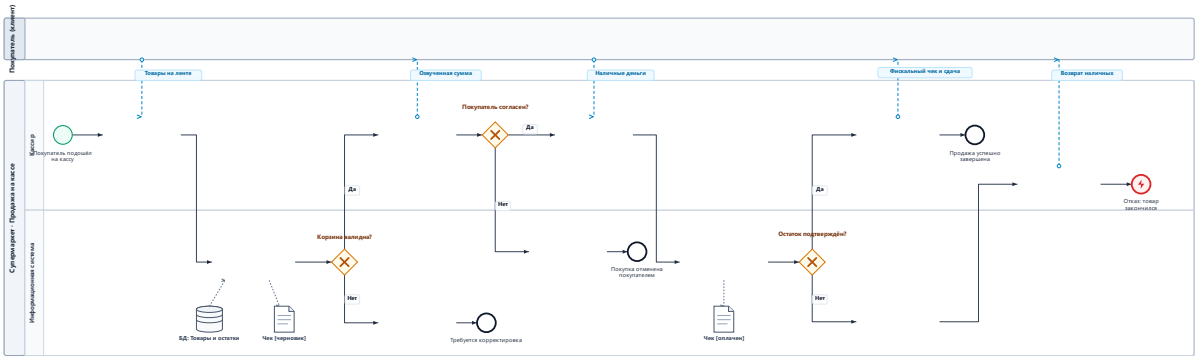



Рисунок 4 — диаграмма бизнес-процесса продажи на кассе (нотация BPMN 2.0)

Модель разработана в соответствии со спецификацией BPMN 2.0 в формате XML и верифицирована средой bpmn-js; исходные файлы моделей передаются в составе электронного архива проекта. Диаграмма опирается на легенду нотации BPMN на с. 26 методички [1] и спецификацию BPMN 2.0.2 [5]: круги — события, скруглённые прямоугольники — задачи, ромбы с X — исключающие шлюзы, дорожки — ответственность кассира и системы, сплошные стрелки — поток управления. Покупатель участвует в процессе, но не имеет интерфейса системы.

Ветка проверки корзины сообщает об ошибке без списания остатков. Если покупатель отказывается, черновик отменяется. Перед оплатой система повторно проверяет остаток под блокировкой и в одной транзакции записывает оплату, чек и движения. При нехватке наличные возвращаются, черновик остаётся доступен для повторной попытки. Диаграмма описывает одну попытку: исправление корзины или повтор оплаты запускают следующий проход с сохранённым черновиком.

Модель процесса сохранена в открытом стандарте BPMN 2.0 XML с сохранением полной структуры пулов, дорожек, событий и развилок. Импорт и отображение проверены официальным инструментарием bpmn-js [7]. Текстовые исходники моделей зафиксированы в составе проекта для обеспечения версионного контроля.

Черновик фиксирует цену, но не резервирует остаток. При отказе оплаты система возвращает причину, кассир отменяет черновик или повторяет проверку. Подтверждение наличных относится к учебному внутреннему учёту и не вызывает банк.

### 3.4 Диаграмма состояний

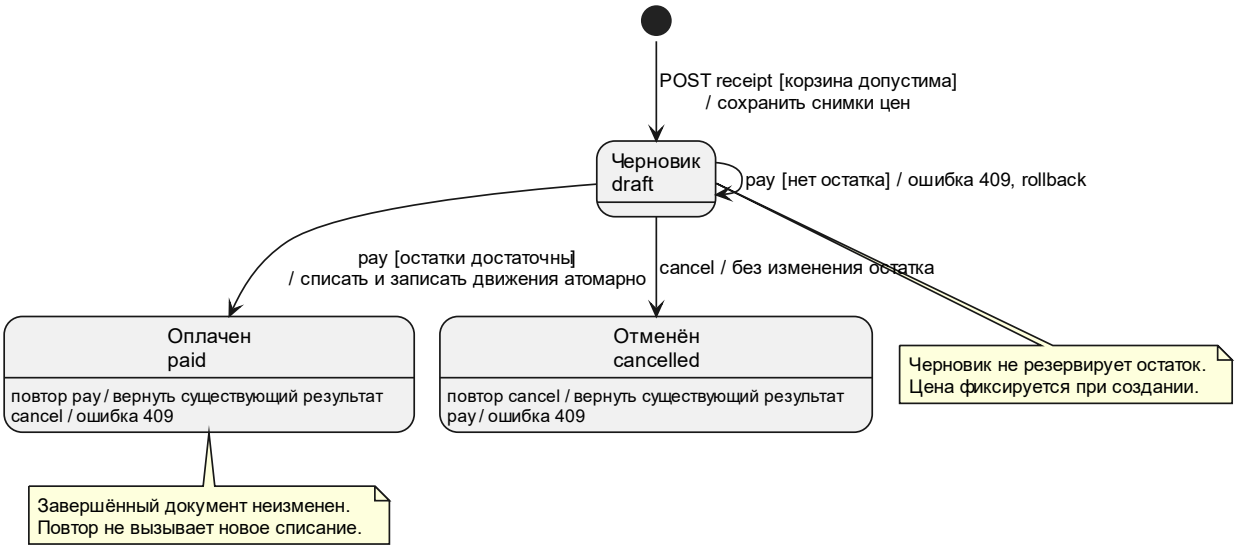



Рисунок 5 — диаграмма состояний

Жизненный цикл сохраняемого чека: создание → draft → paid либо cancelled. Повтор оплаты paid и повтор отмены cancelled возвращают существующий документ без изменения состояния и без повторных побочных эффектов. Ошибка нехватки оставляет draft. Несохранённая корзина относится к UI и не является строкой таблицы чеков.

### 3.5 ER-модель

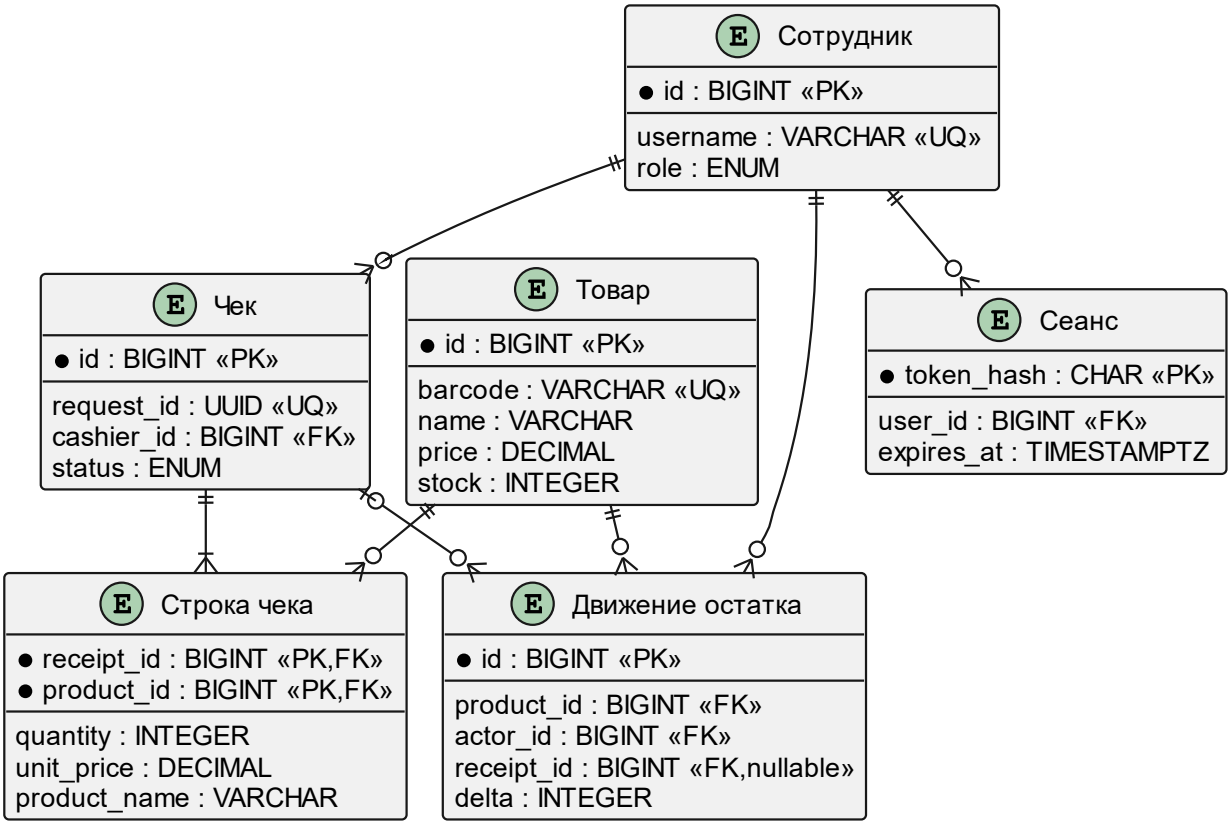



Рисунок 6 — ER-диаграмма

Связи: сотрудник 1:N чек; чек 1:N строка; товар 1:N строка; товар 1:N движение; сотрудник 1:N движение; чек 0:N движение; сотрудник 1:N сеанс. Связь товар–чек M:N разрешается строкой чека. В строке хранятся исторические реквизиты — осознанная денормализация для неизменности истории. Текущий остаток в товаре ускоряет продажу, а движения объясняют изменения.

### 3.6 Заключение и выводы

1. UC-модель задаёт действия, объектная модель — конкретный снимок, ER — структуру, BPMN — процесс, состояния — допустимые переходы одного документа.

2. Согласованность обеспечивается одинаковыми названиями статусов и бизнес-правилами R01–R12 во всех моделях.

3. Внешние платежи не моделируются как реализованная интеграция.

### 3.7 Проверка машинной модели

Каноническая модель бизнес-процесса в формате BPMN 2.0 XML дополнена полной диаграммной разметкой (BPMN DI): взаимодействие пулов, две горизонтальные дорожки ответственности («Кассир» и «Информационная система»), 17 узлов процессов, потоки управления и сообщений. Корректность синтаксиса и визуализации подтверждена валидатором bpmn-js 18.28.0. Векторные и исходные файлы диаграмм передаются в составе электронного архива проекта. При отмене покупки наличные ещё не принимались; при отказе атомарной оплаты после получения наличных кассир возвращает их, а черновик и остаток не изменяются.

# 4 Архитектура программного обеспечения

## 4.1 Вариант архитектуры и обоснование

Выбрана трёхуровневая клиент-серверная архитектура: React-клиент в браузере, FastAPI-сервер приложений, PostgreSQL. Nginx отдаёт статические файлы и проксирует /api. Предметные функции объединены в модульный монолит: для одной команды из трёх человек это позволяет сохранять продажу одной транзакцией и не вводить распределённые транзакции. Стек соответствует согласованному отступлению от примеров методички; официальная документация React, FastAPI и PostgreSQL использована как технический справочник [9–11].

### 4.1.1 Модульная структура

Таблица 8 — Модули, методы и данные проектируемой системы

| **Модуль** | **Назначение и методы в псевдокоде** | **Данные и зависимости** |
|---|---|---|
| Auth | login(credentials), me(token), logout(token), requireRole(role) | Сотрудник, сеанс; использует Security и DB |
| Security | hashPassword(password), verifyPassword(password, hash), hashToken(token) | Соль и хеш; не зависит от UI |
| Catalog | list(query,cursor,limit), create(card), update(card,version) | Товар; использует Auth, Validation, DB |
| Inventory | adjust(product,delta,reason), recordMovement(...) | Остаток и движение; использует Auth, DB |
| Sales | createReceipt(key,lines), pay(id), cancel(id), read(id), history(cursor) | Чек, строка; использует Auth, Catalog/Inventory через общую транзакцию |
| Validation | validateProduct(...), validateReceipt(...) | Схемы запросов; ограничивает типы, диапазоны и повтор товаров |
| DB | acquireConnection(), begin(), commit(), rollback() | Пул соединений и параметризованные SQL-запросы |
| Web Catalog | search(), nextPage(), addToCart(), editCard() | Состояние фильтра, текущая страница; вызывает API-client |
| Web Checkout | prepare(), confirmCash(), cancelDraft() | Корзина, серверный чек, ключ повтора |
| API-client | request(path,body,token), validateResponse(...) | JSON, отмена запроса, преобразование ошибок |

Декомпозиция по функциям; наследование не служит основным способом организации приложения. Схемы запросов могут наследовать общие правила Pydantic. React-компоненты составляются композиционно и используют хуки.

### 4.1.2 Компоненты и соединители

Браузер инициирует HTTP-запросы; Nginx отдаёт сборку и передаёт API-запросы FastAPI. Сервер выполняет аутентификацию, валидацию и бизнес-правила, затем обращается к БД по PostgreSQL wire protocol. Совместно используемые данные находятся только в PostgreSQL. UI не открывает соединение с БД.

Параллелизм: несколько браузеров и несколько обработчиков API используют общий пул. При оплате сервер блокирует чек, затем товары в возрастающем порядке ID. Такой порядок уменьшает возможность взаимоблокировки. Строки разных товаров допускают параллельные продажи; конфликт за последний экземпляр разрешается БД. READ COMMITTED с блокировками достаточен для выбранной транзакции; повторная проверка после получения блокировки обязательна.

Псевдокод оплаты: проверить сеанс → начать транзакцию → заблокировать чек → вернуть результат, если уже paid → проверить draft → заблокировать товары по ID → проверить остаток и активность → уменьшить остаток → создать движения → пометить paid → commit. Ошибка на любом шаге приводит к rollback. Сетевой ответ отправляется после фиксации транзакции.

## 4.2 Структуры распределения

### 4.2.1 Размещение

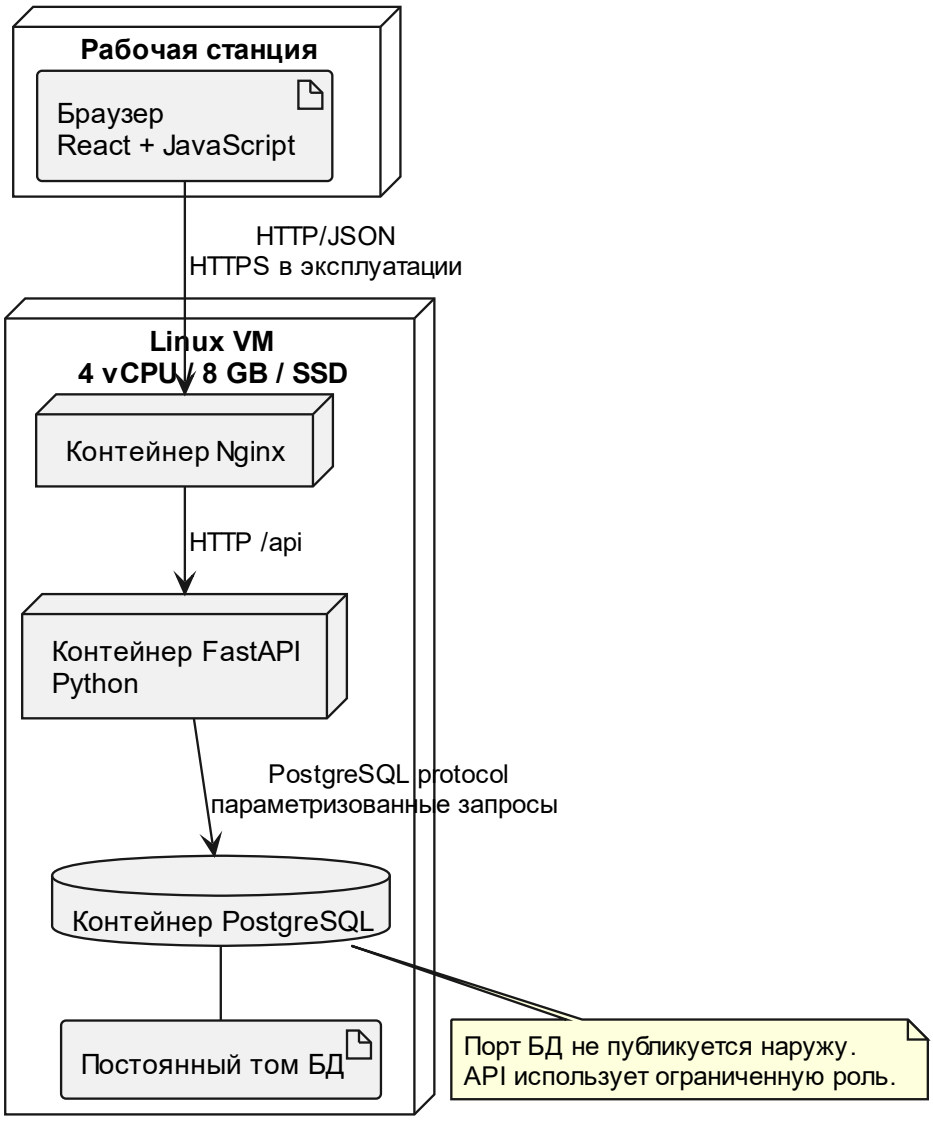



Рисунок 7 — схема физ. размещения компонентов системы

Рабочая станция кассира выполняет браузер. Linux VM выполняет Nginx, FastAPI и PostgreSQL в отдельных контейнерах. Том БД сохраняется между перезапусками. База и API не публикуют порты на внешний интерфейс; доступ к приложению в учебном стенде ограничен localhost. Для доступа из локальной сети адрес публикации и TLS настраиваются отдельно.

Проектная VM: 4 vCPU, 8 ГБ RAM, 40 ГБ диска с возможностью расширения; гостевая ОС Linux, сеть NAT с ограниченным пробросом или host-only. Производительность проверяется измерением на реальном

выделенном стенде. Контейнеры не заменяют этап установки VM, предусмотренный ЛР №2.

### 4.2.2 Реализация

Таблица 9 — План отображения модулей на файлы будущей реализации

| **Модуль** | **Планируемое место реализации на этапе ЛР №3** |
|---|---|
| Сборка приложения и lifecycle | backend/app/main.py |
| HTTP-маршруты Auth, Catalog, Sales | backend/app/routers/auth.py, products.py, receipts.py |
| Предметные операции Catalog, Inventory, Sales | backend/app/services/products.py, receipts.py |
| Аутентификация и проверка ролей | backend/app/dependencies.py |
| Преобразование ошибок в HTTP-ответы | backend/app/errors.py |
| Validation | backend/app/schemas.py |
| Security | backend/app/security.py |
| DB | backend/app/db.py |
| Схема и права БД | init.sql, db/pg_hba.conf |
| Web Catalog | frontend/src/Catalog.jsx |
| Оркестрация состояния интерфейса | frontend/src/App.jsx |
| Вход, навигация, редактор товара | frontend/src/components/*.jsx |
| Checkout и история | frontend/src/features/checkout/*.jsx, frontend/src/features/receipts/*.jsx |
| API-client | frontend/src/api.js |
| Стили | frontend/src/styles.css |
| Развёртывание прикладной версии | docker-compose.yml, Dockerfile компонентов |

Это план структуры будущего кода. В ЛР №1 размещаются только документы и модели. ЛР №2 создаёт отдельный инфраструктурный стенд с заглушкой и своим томом; прикладная схема и исходники появляются только в ЛР №3. ЛР №4 реализует отдельную инструментальную утилиту варианта 12.

### 4.2.3 Распределение функций между участниками

Таблица 10 — Обязанности по архитектуре и интеграции

| Участник | Роль в ЛР №1 | Ответственность | Результат и форма отчетности |
|---|---|---|---|
| A | Специалист по НИОКР | Концепция, требования, ограничения и критерии | Разделы 1 и 2 отчёта (концепция, словарь данных, спецификация требований BR/FR/NFR) |
| B | Системный аналитик | UC, объекты, BPMN, состояния, ER; согласование терминов | Раздел 3 отчёта (диаграммы Use Case, объектов, состояний, ER-модель, процесс BPMN) |
| C | Архитектор ПО | Модули, протоколы, транзакции, размещение и план реализации | Раздел 4 отчёта (модули, компоненты и соединители, структуры размещения и план реализации) |

На этапе реализации A принимает GUI, B реализует API и интеграцию, C отвечает за БД. A — Кочетков Роман; B — Энгельгард Кирилл; C — Прохоров Максим. Обмен контрактом JSON и согласование бизнес-правил предшествуют реализации; изменения требований оформляются задачей и проходят совместный обзор.

## 4.3 Надёжность и эксплуатационные границы

Блокировки обеспечивают согласованность между экземплярами API, а не только внутри одного процесса. Размер пула ограничивает число соединений. Запросы имеют таймаут; занятый сервис отвечает временной ошибкой, которую клиент может повторить. Отсутствие сети не разрешает клиенту списывать остатки самостоятельно.

Секреты передаются через локальный конфигурационный файл, не включаются в сборку клиента и Git. Различаются пользовательские роли приложения и технические роли БД. Администратор БД доверенный и может обходить прикладные ограничения; его действия выполняются по регламенту. Контур приложения и контур Redmine имеют разные базы и тома.

### 4.4 Заключение и выводы

1. Три уровня разделяют отображение, принятие решений и сохранение данных.
2. Модульный монолит соответствует размеру команды и общей транзакции продажи.
3. Структуры размещения, реализации и распределения обязанностей описывают разные стороны одной архитектуры.
4. Масштабирование начинается с корректных запросов, индексов и ограниченных ответов; отдельные сервисы вводятся только при подтверждённой необходимости.

## 5 Роли и защита ЛР №1

Методичка, с. 43: A — специалист по научно-исследовательским и опытно-конструкторским разработкам (концепция и требования); B — системный аналитик (модель предметной области); C — архитектор ПО (архитектура). Приоритет выступлений: A, B, C. A — Кочетков Роман; B — Энгельгард Кирилл; C — Прохоров Максим. Группа, преподаватель, даты и подписи заполняются при оформлении сдачи.

Каждому подготовить доклад на 3–5 минут: постановка задачи, собственный результат, проблема и принятое решение, вывод. A объясняет конфликты цен/остатков и источники постановки. B объясняет нормализацию связей, сохранение снимков цены, отличие BPMN от диаграммы состояний. C объясняет три уровня, транзакцию оплаты и три структуры распределения.

Контрольные ответы: актор — внешняя роль по отношению к границе системы; UC — цель взаимодействия; объект — конкретный экземпляр; состояние — допустимый этап жизни объекта; конфликт требований обнаруживается сопоставлением моделей и CRUD; архитектура определяет компоненты и связи, а структура реализации — отображение модулей на файлы.

Источники и нормативная основа: методичка ЛР №1 (2021), включая рисунок BPMN на с. 26; ISO/IEC/IEEE 29148:2018 для требований; UML 2.5.1 и BPMN 2.0.2 от OMG для моделей; официальное руководство PlantUML и bpmn-js для исходников и отрисовки. Полные библиографические описания стандартов и нормативных источников приведены в разделе «СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ» данного отчёта. Предметные предположения сформулированы для учебного супермаркета и ограничены концепцией.

# ЗАКЛЮЧЕНИЕ

Сформированы концепция, словарь, конфликты, дерево функций, спецификация и CRUD-матрица. Подготовлены UML-модели, BPMN и ER-модель. Определены клиент-серверные компоненты, правила транзакций и структуры размещения, реализации и распределения обязанностей. Модели проверены программной отрисовкой; первая работа сохранена как самостоятельный этап проектирования.

## СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Основы программной инженерии. Методические указания к выполнению лабораторной работы № 1 / сост. П. А. Охтилев, А. Э. Зянчурин. Санкт-Петербург : ГУАП, 2021. Электронный документ, предоставленный к заданию.

2. ГУАП. Для учебного процесса. Нормативная документация. URL: https://guap.ru/c/regdocs/docs/uch (дата обращения: 03.10.2026).

3. ISO/IEC/IEEE 29148:2018. Systems and software engineering — Requirements engineering. URL: https://www.iso.org/standard/72089.html (дата обращения: 03.10.2026).

4. OMG. Unified Modeling Language (UML), Version 2.5.1. URL: https://www.omg.org/spec/UML/2.5.1/ (дата обращения: 03.10.2026).

5. OMG. Business Process Model and Notation (BPMN), Version 2.0.2. URL: https://www.omg.org/spec/BPMN/2.0.2/PDF (дата обращения: 03.10.2026).

6. PlantUML. Language Reference Guide. URL: https://plantuml.com/guide (дата обращения: 03.10.2026).

7. bpmn-io. bpmn-js: BPMN 2.0 rendering toolkit and web modeler. URL: https://github.com/bpmn-io/bpmn-js (дата обращения: 03.10.2026).

8. n8n. Documentation: workflow automation and integrations. URL: https://docs.n8n.io/ (дата обращения: 03.10.2026).

9. React. Learn React. URL: https://react.dev/learn (дата обращения: 03.10.2026).

10. FastAPI. Tutorial. URL: https://fastapi.tiangolo.com/tutorial/ (дата обращения: 03.10.2026).

11. PostgreSQL Global Development Group. PostgreSQL 17 Documentation. URL: https://www.postgresql.org/docs/17/ (дата обращения: 03.10.2026).